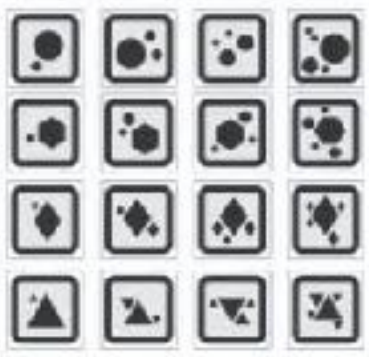
Laboratoire 2 : Apprendre à reconnaître des objets

Maintenant que notre agent sait explorer son environnement, il est prêt à apprendre. Vous avez déjà pu observer ses capacités de perception lors du parcours précédent. Dans ce laboratoire, nous allons lui apprendre à reconnaître des formes simples. Pour cela, nous utiliserons une banque d'images créée par les étudiants des années précédentes, contenant des marqueurs ArUco — des symboles visuellement proches des codes QR.

L'objectif de ce laboratoire est de concevoir et de comparer différents modèles d'apprentissage automatique pour la reconnaissance d'objets simples. Vous commencerez par constituer une base de données d'images adaptée à l'entraînement et à l'évaluation des modèles, puis vous comparerez leurs performances respectives.

Jeu de données:

Vous utiliserez les images fournies dans le fichier « **EnsembleDeDonnéesA** ». Cet ensemble contient plusieurs photos pour chaque type de marqueur ArUco. Au total, 16 symboles différents sont représentés, comme illustré ci-dessous.



Objectifs Spécifiques

1. Préparation des ensembles de données

- À partir de **l'EnsembleA**, préparez un **sous-ensemble** comportant plusieurs exemples couvrant **au minimum huit (8) symboles distincts**. Ce sous-ensemble sera nommé **EnsembleB**.
  ➤ **Justifiez le choix** des symboles inclus dans EnsembleB. Pourquoi avoir choisi ces derniers? Quel est l'avantage de ne choisir que 8 symboles ou au contraire, pourquoi en avoir choisi plus?

2. Prétraitement des images

- *(Optionnel – mais recommandé, voir point 3)* : **Réduisez la taille des images** (par exemple de 320×140 à 160×70).

- **Extraction de la région d'intérêt (ROI)** : isolez la zone contenant le marqueur, soit **manuellement**, soit **automatiquement** à l'aide d'outils de la bibliothèque **OpenCV** (par exemple un algorithme de détection de formes).

3. Altérations d'images *(optionnel)*

Si vous appliquez des transformations aux images, **détaillez clairement** les altérations effectuées.
Exemples d'altérations possibles :

- **Flou** : pour atténuer les détails et améliorer la détection des formes.
- **Filtre en niveaux de gris** : pour réduire la complexité et le nombre de primitives.
- **Augmentation de données** : appliquez des rotations, translations, distorsions ou du bruit pour enrichir votre ensemble d'entraînement.

4. Extraction des caractéristiques

- Utilisez **les pixels** de la région d'intérêt pour représenter les images.
- *(Optionnel)* : ajoutez d'autres caractéristiques, par exemple :
  - le **nombre de formes** dans l'image,
  - l'**excentricité** de la plus grande forme, etc.

5. Implémentation d'un réseau de neurones

- Implémentez **un réseau de neurones complet** comportant **au moins une couche cachée**.
- **N'utilisez pas d'implémentation existante (aucune librairie prête à l'emploi).**
- L'implémentation doit être **vectorisée** (aucune boucle for).

6. Évaluation du modèle

- Évaluez les performances de votre réseau à l'aide de **l'EnsembleB**.
- Utilisez soit :
  - une **validation croisée** à *k* échantillons (*k = 3 ou 5*),
  - ou une **méthode hold-out** (séparation entraînement / validation / test).
    ➤ **Justifiez le choix** de votre méthode et de vos **hyperparamètres**.

7. Comparaison avec d'autres modèles

- Évaluez également trois modèles disponibles dans **Scikit-learn** :
  - **SVM**
  - **k-NN**
  - **RN (réseau de neurones)**
- Il est **recommandé** de normaliser les caractéristiques pour les deux implémentations de réseaux de neurones.
  ➤ Vous pouvez utiliser **MinMaxScaler** ou **StandardScaler** de *Scikit-learn*.

8. Tests de scalabilité

- Testez la **scalabilité** des modèles en réduisant la taille des données d'entraînement à
  80% et 30%.
- Observez l'impact sur les performances.

9. Comparaison et présentation des résultats

Présentez les résultats sous forme claire et structurée :

- **Tableau** : précision, rappel, F-mesure
- **Temps d'entraînement** (graphe ou tableau comparatif)
- **Temps de prédiction** (graphe ou tableau comparatif)
- **Résultats de scalabilité**

Remise

Vous devez remettre dans un .zip:

- L'ensemble des images modifiées constituant votre base de données
- Un fichier pour jupyter notebook contenant votre script. Ce script doit être commenté. Le premier commentaire devrait indiquer le « absolute path » (la position) du script sur le robot.
- Un court rapport contenant les résultats des comparaisons de modèles.

Critères	Total	Points
Ensemble de données	**2**	
Avoir au moins 8 classes	0.5	
Extraction des caractéristiques à partir des images (pixels) : • Remise d'un fichier ZIP contenant les photos prétraitées, classée par étiquette	1.5	
Réseau de neurones	**8**	
Implémentation de la propagation avant from scratch	1.5	
Implémentation de la propagation arrière from scratch	1.5	
Utilisation d'une technique de validation	1	
Code PY complet fourni, fonctionnel et commenté	1	
Implémentation du KNN	1	
Implémentation du SVM	1	
Implémentation du RN (avec librairie)	1	
RAPPORT	**5**	
Justification des images sélectionnées et des retraitements faits, si applicables.	0.5	

Description de l'implémentation de votre réseau de neurones: • Type de réseau (0.5) • Nombre de couches cachées (0.5) • Fonction d'activation (0.5) • Choix des hyperparamètres (0.5)	2	
Comparaison des résultats du réseau de neurones aves les 3 autres modèles: • Tableau précision, rappel, f-mesure (0.5) • Résultats des tests de scalability (0.5) • Temps nécessaire pour l'entrainement (0.5) • Temps nécessaire pour les prédictions (0.5)	2	
Interprétation des résultats et conclusion	0.5	
TOTAL SUR 15	**15**	
NOTE FINALE	**15%**	

Critères négatifs :

- Mauvaise qualité du code (lisibilité) : - [0...10] %
- Non-respect des critères de remises : - [0..10] %
- Mauvaise qualité des explications et présentation du rapport : - [0..10] %